



USC University of
Southern California

Lab 11 - Hash Tables

CS 104

Last Updated: 03/29/2026

Hash Tables: Fast Lookups in Practice

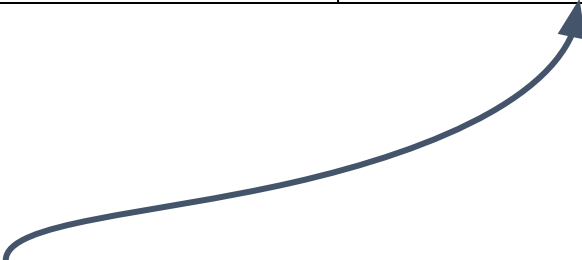
USC

The Problem

We want fast lookups ... but everything has tradeoffs

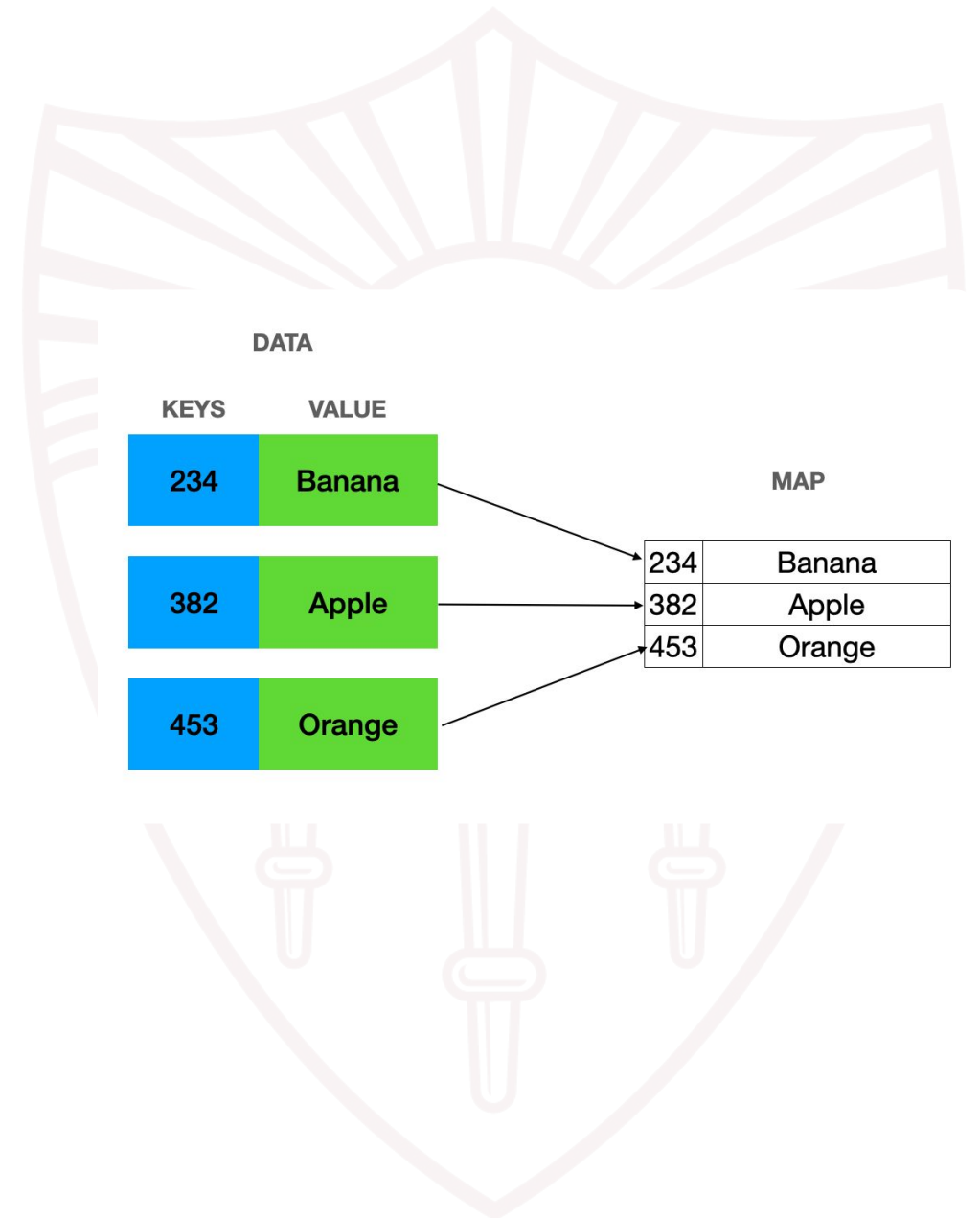
Structure	Lookup	Insert
Array	$O(1)$	$O(n)$
Linked List	$O(n)$	$O(1)$
BST	$O(\log n)$	$O(\log n)$

Hash tables try to cheat this tradeoff



What is a Map

- Key → Value
- Keys are unique
- Used for **searching / lookup**

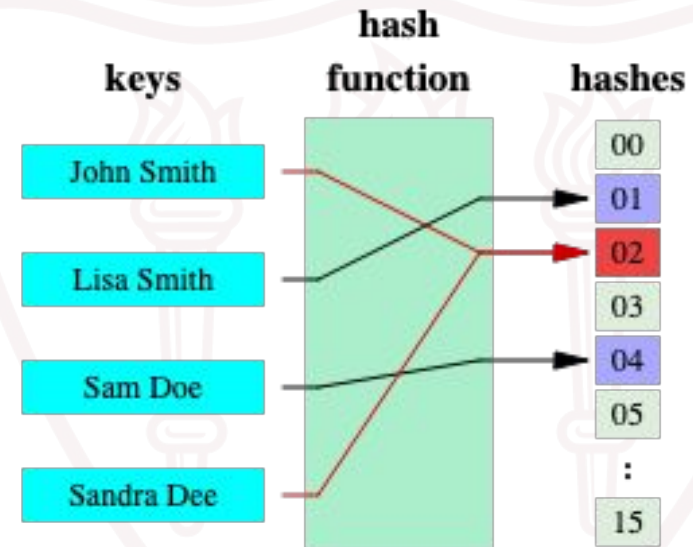


Big Idea of Hash Tables

Like an array, but to find the index we use a hash function

- Use a **hash function**
- Directly jump to index
- No traversal

Idea: compute where data goes instead of searching



Big Idea of Hashing

Array: [?, ?, ?, ?, ?, ?, ?, ?, ?, ?]

Hmmm I wonder if
“John Smith” is
somewhere in this
array



Big Idea of Hashing

Array: [?, ?, ?, ?, ?, ?, ?, ?, ?, ?]

How do I find
it?



Big Idea of Hashing

Array: [?, ?, ?, ?, ?, ?, ?, ?, ?, ?]



Searching ...



Big Idea of Hashing

Array: [?, ?, ?, ?, ?, ?, ?, ?, ?, ?]



Searching ...



Big Idea of Hashing

Array: [?, ?, ?, ?, ?, ?, ?, ?, ?, ?]



Searching ...



Big Idea of Hashing

Array: [?, ?, ?, ?, ?, ?, ?, ?, ?, ?]



Searching ...



Big Idea of Hashing

Array: [?, ?, ?, ?, ?, ?, ?, ?, ?, ?]



Searching ...



Big Idea of Hashing

Array: [?, ?, ?, ?, ?, ?, ?, ?, ?, ?]



Searching ...



Big Idea of Hashing

Array: [?, ?, ?, ?, ?, ?, ?, ?, ?, ?]



Searching ...



Big Idea of Hashing

Array: [?, ?, ?, ?, ?, ?, ?, ?, ?, ?]



Searching ...



Big Idea of Hashing

Array: [?, ?, ?, ?, ?, ?, ?, ?, ?, ?]



Searching ...



Big Idea of Hashing

Array: [?, ?, ?, ?, ?, ?, ?, ?, ?, ?]



Searching ...



Big Idea of Hashing

Array: [?, ?, ?, ?, ?, ?, ?, ?, ?, ?]

We had to search every. single. item.
SO SLOW AHHHHH



Big Idea of Hashing

Array: [?, ?, ?, ?, ?, ?, ?, ?, ?]

What if I had a function that takes a key and tells me exactly where it lives?



Big Idea of Hashing

Array: [?, ?, ?, ?, ?, ?, ?, ?, ?]

A HASH FUNCTION!

```
hash( "John Smith" )
```

FOUND IT!!



Big Idea of Hashing

TL;DR – With hashing we can instantly find in an array where an item belongs in $O(1)$

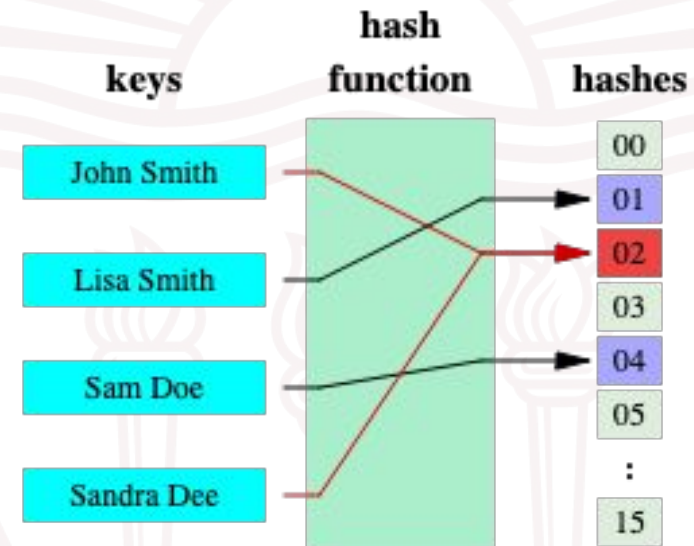
Every key computes its own location



Hash Table – A Magical Array

- Under the hood: array
- Index = hash(key)
- Average lookup: **$O(1)$**

Hash function: how this magic works – maps each key to an index so we can do instant lookup

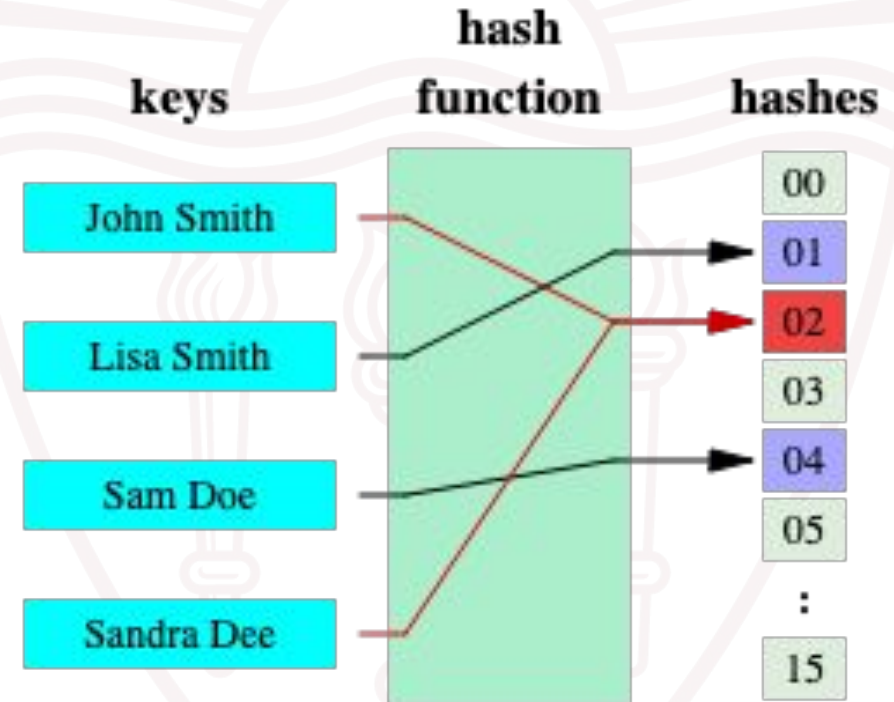


Hash Function (Intuition)

Goal: spread data evenly

Good hash:

- Fast
- Deterministic
- Uniform distribution



Good vs. Bad Hash

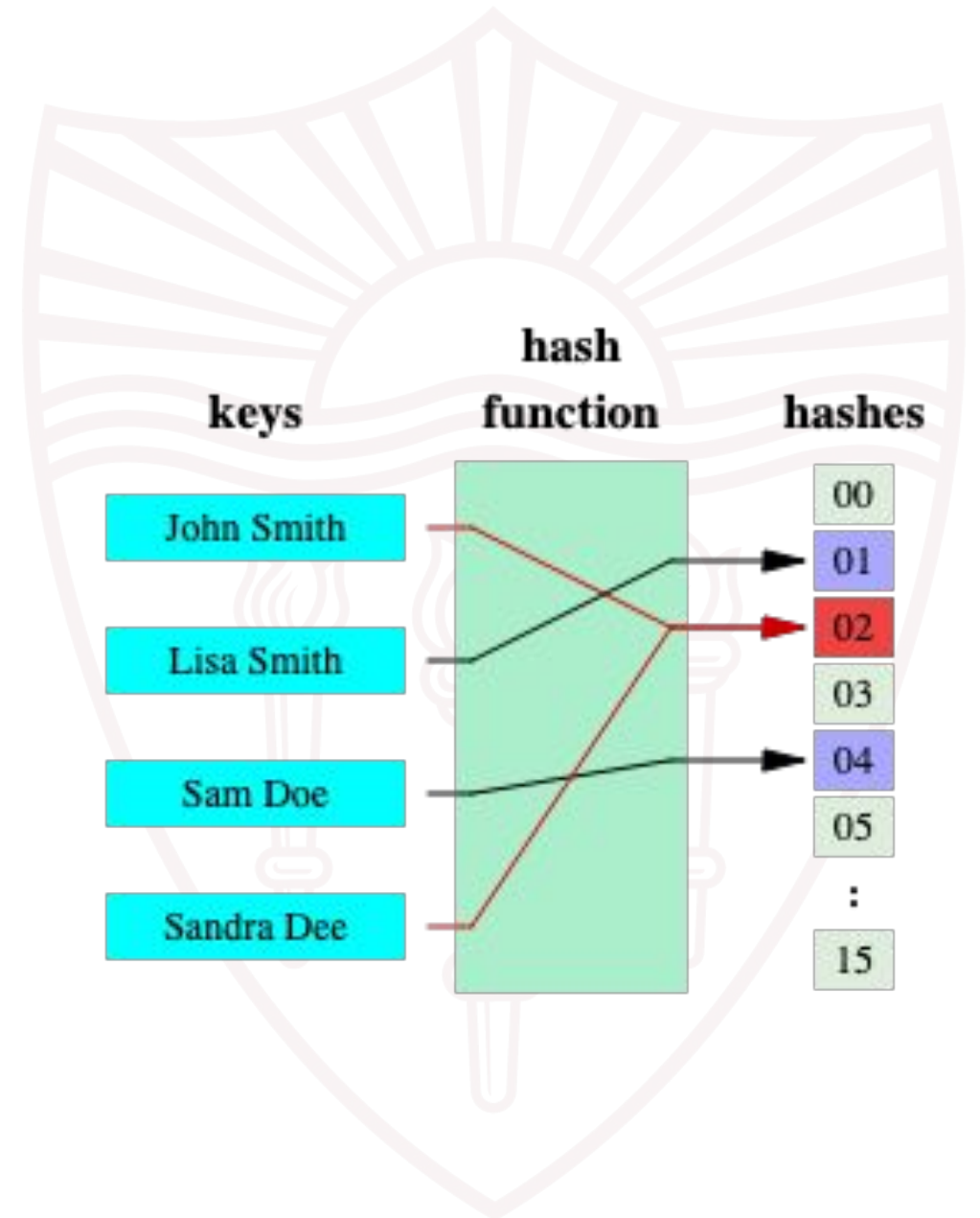
Goal: spread data evenly

Bad:

everything → same index

Good:

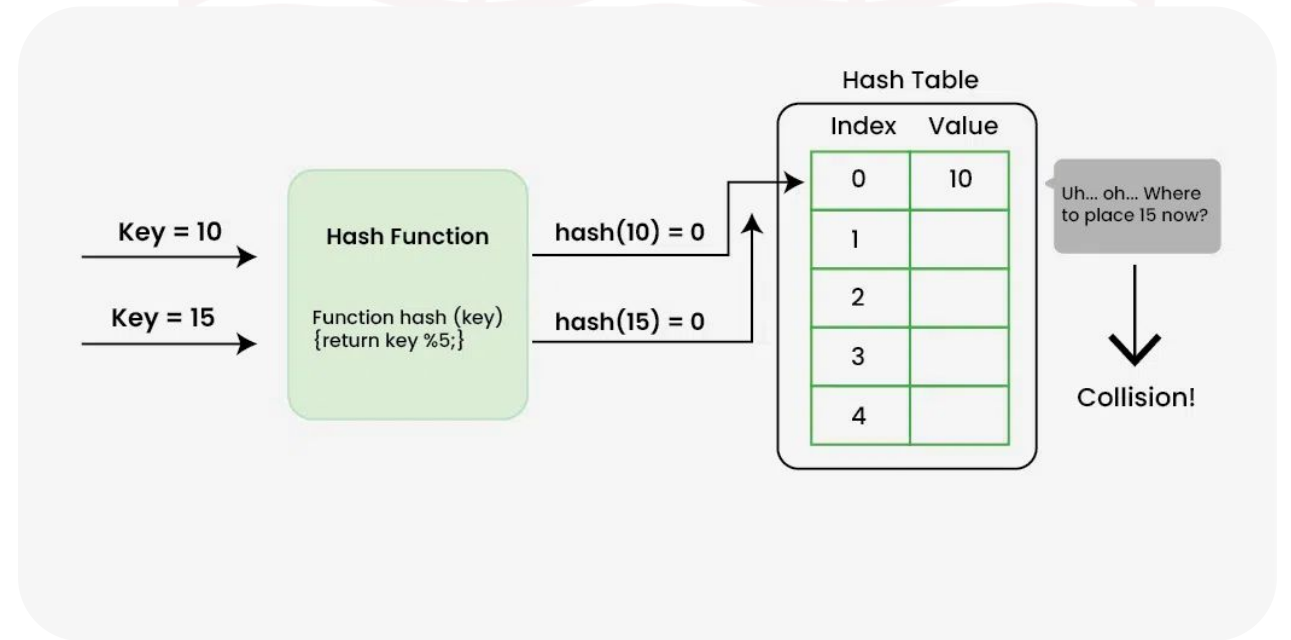
spreads keys across table



The Problem: Collisions

A collision: two keys are mapped to the same address

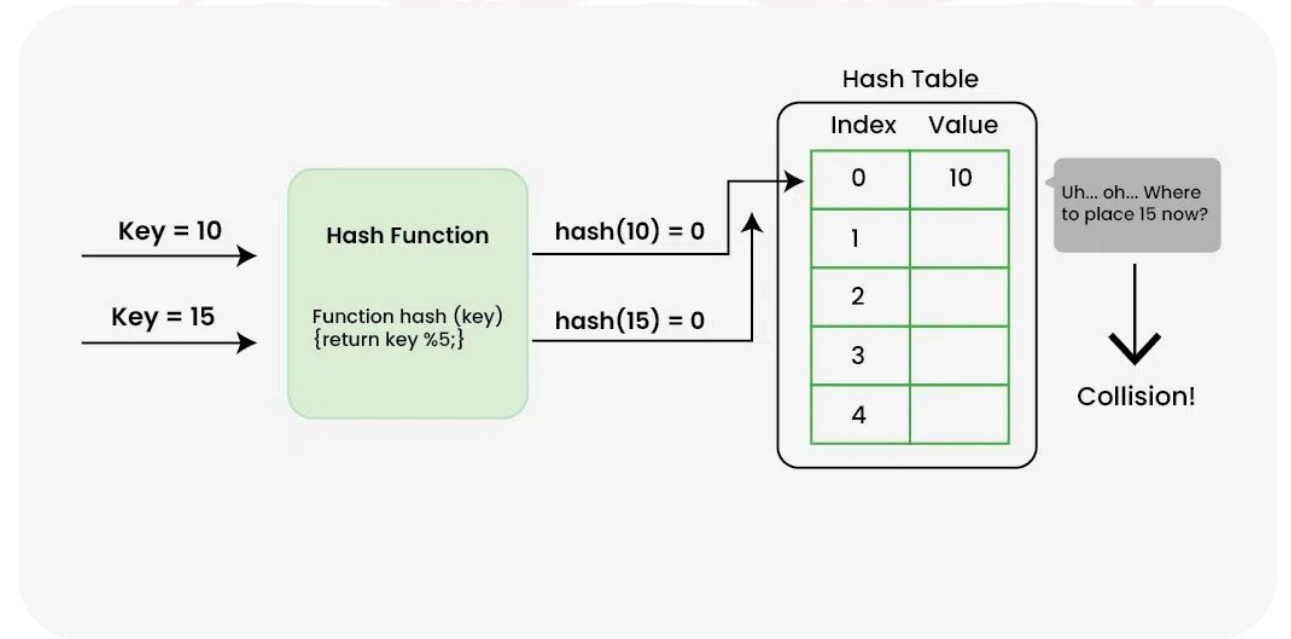
Hashing isn't perfect —
collisions are guaranteed



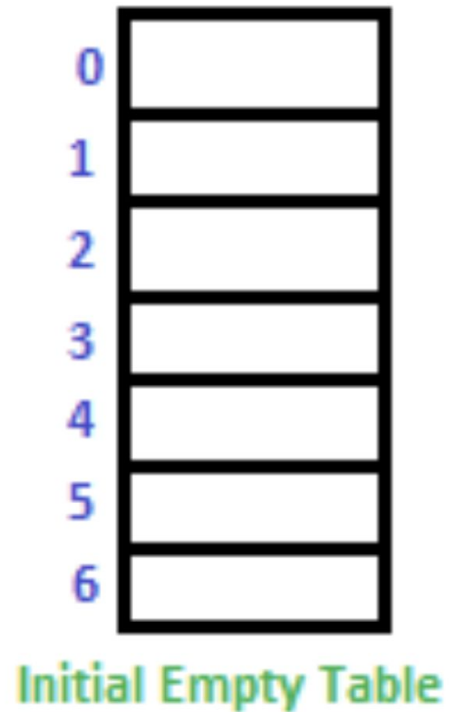
Two Ways to Fix Collisions

Everything from here is just “how do we handle collisions?”

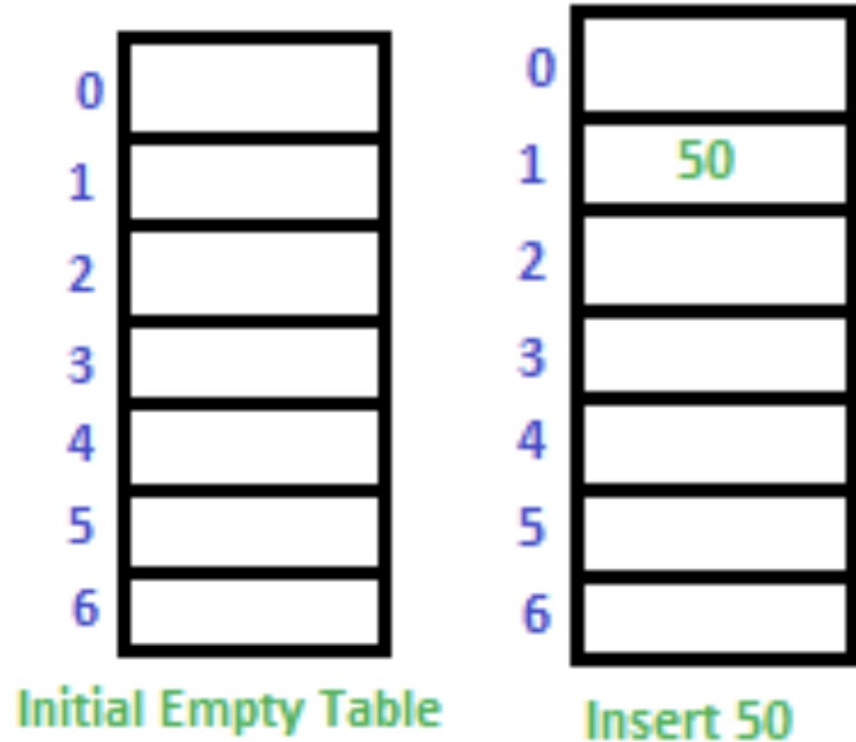
1. Open Addressing
2. Chaining



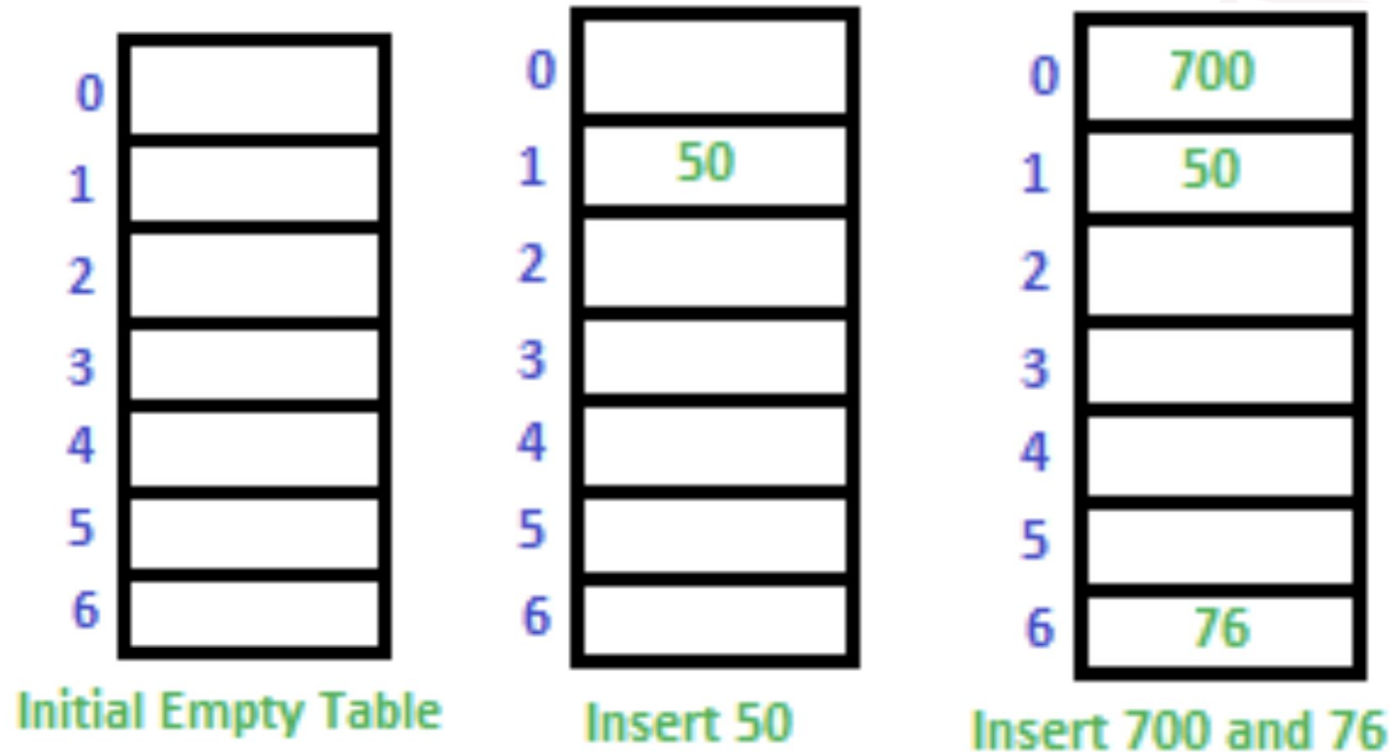
Fixing Collisions: Open Addressing



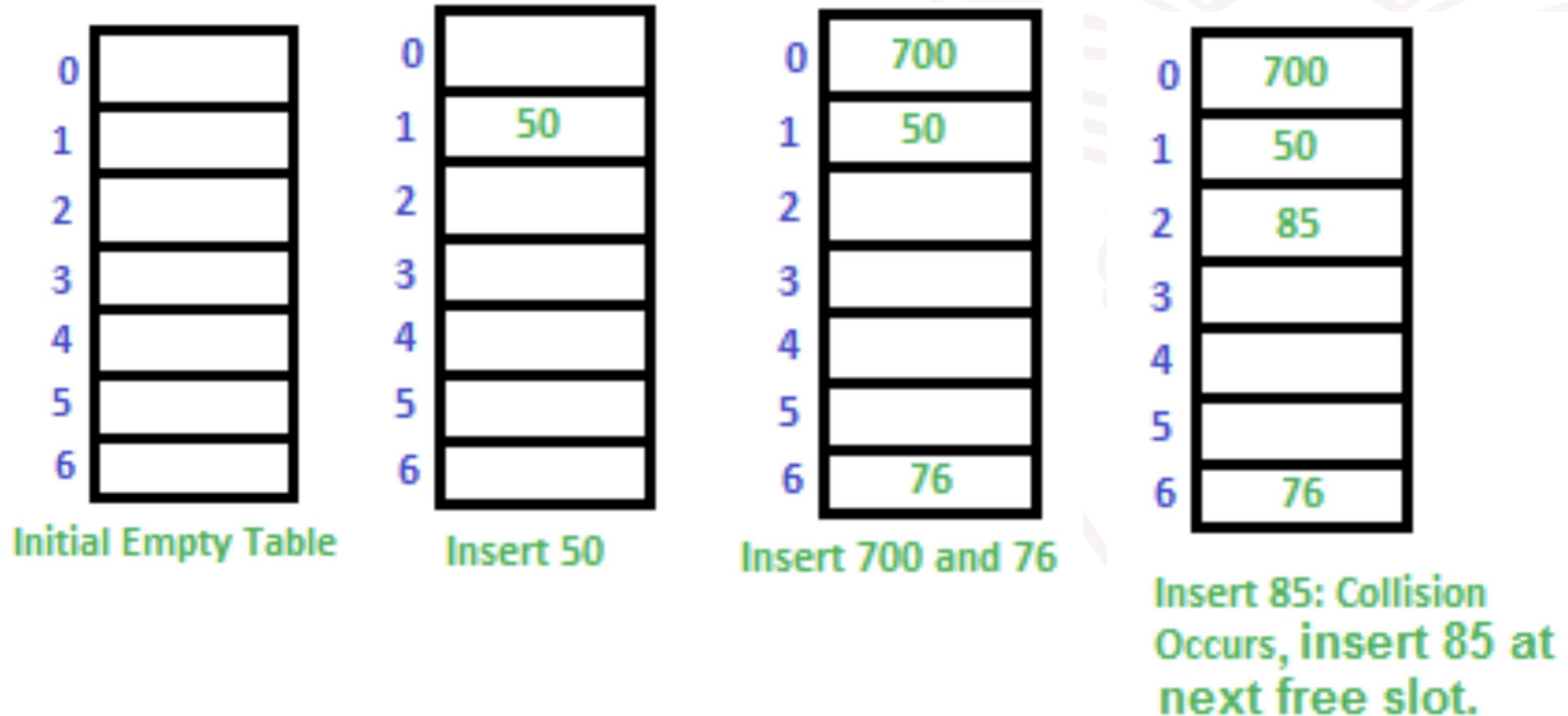
Fixing Collisions: Open Addressing



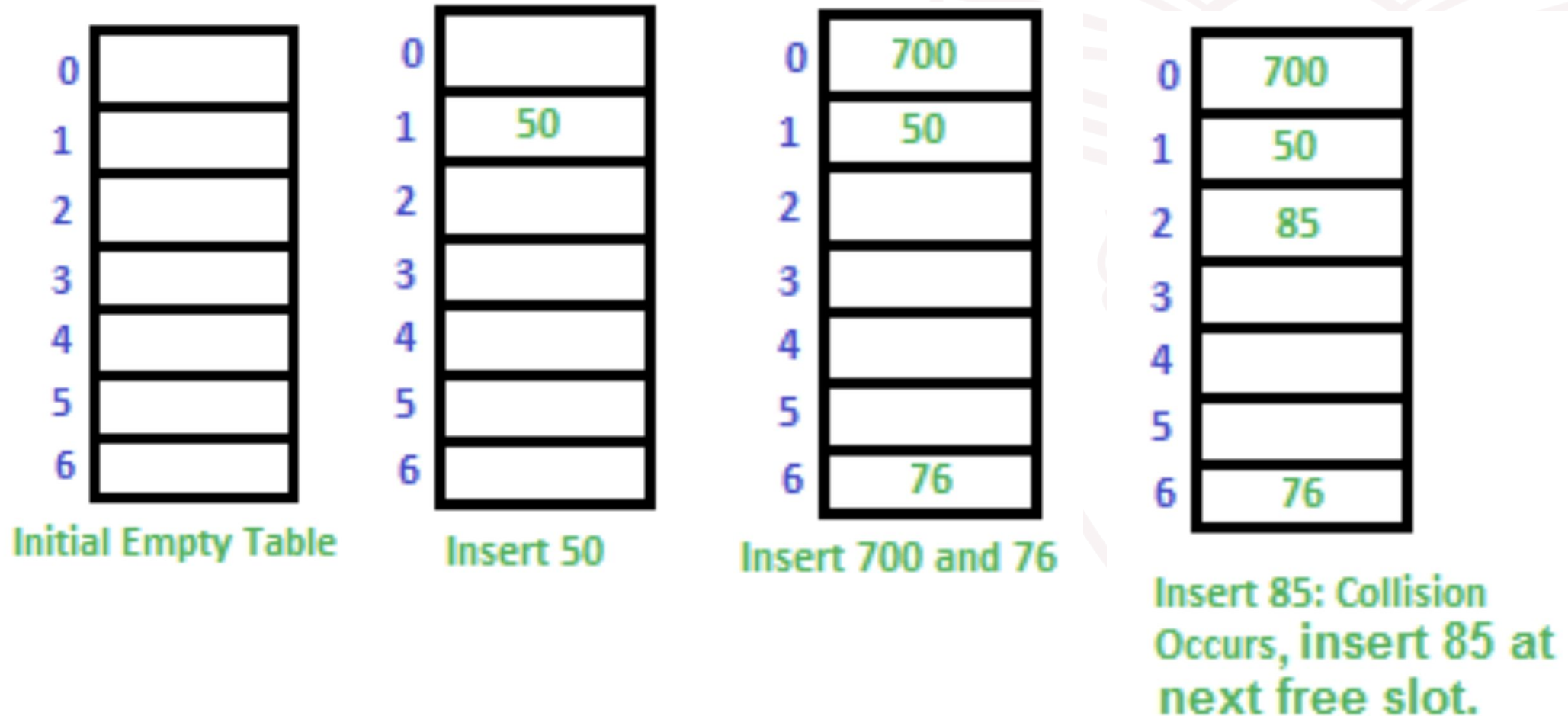
Fixing Collisions: Open Addressing



Fixing Collisions: Open Addressing



Fixing Collisions: Open Addressing



Fixing Collisions: Open Addressing

0	700
1	50
2	85
3	92
4	
5	
6	76

Insert 92, collision occurs as 50 is there at index 1.
Insert at next free slot

Fixing Collisions: Open Addressing

0	700
1	50
2	85
3	92
4	
5	
6	76

Insert 92, collision occurs as 50 is there at index 1.
Insert at next free slot

0	700
1	50
2	85
3	92
4	73
5	101
6	76

Insert 73 and 101

Fixing Collisions: Open Addressing

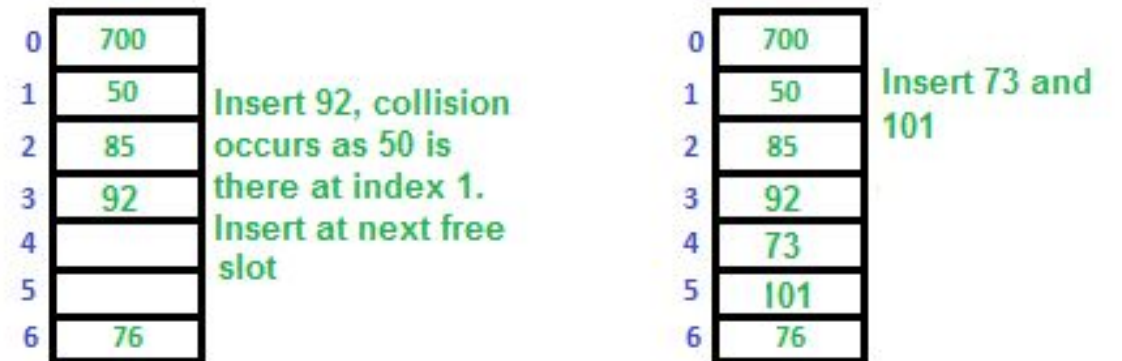
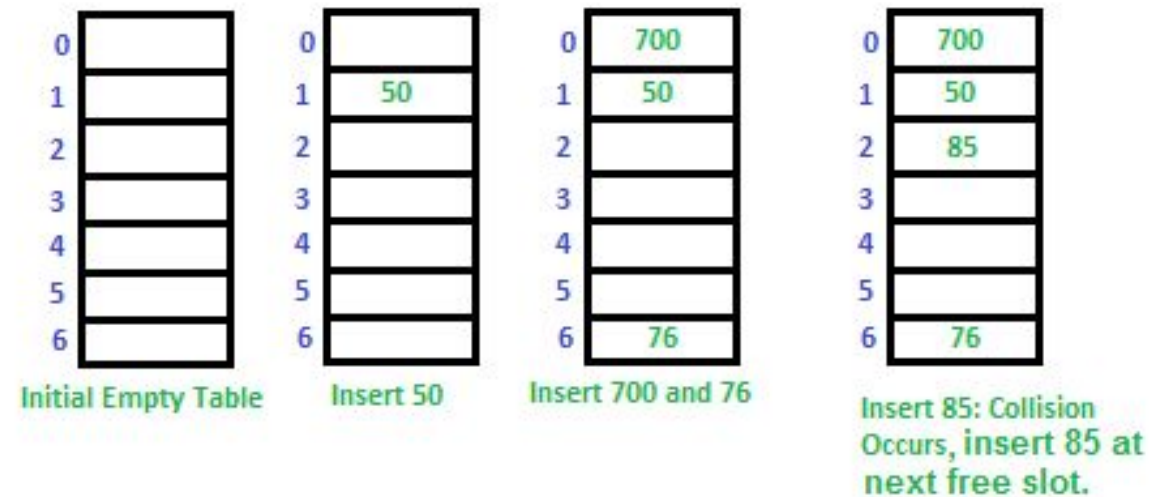
- One item per slot
- If full → probe next

Linear probing:

→ go to next index

Other Examples:

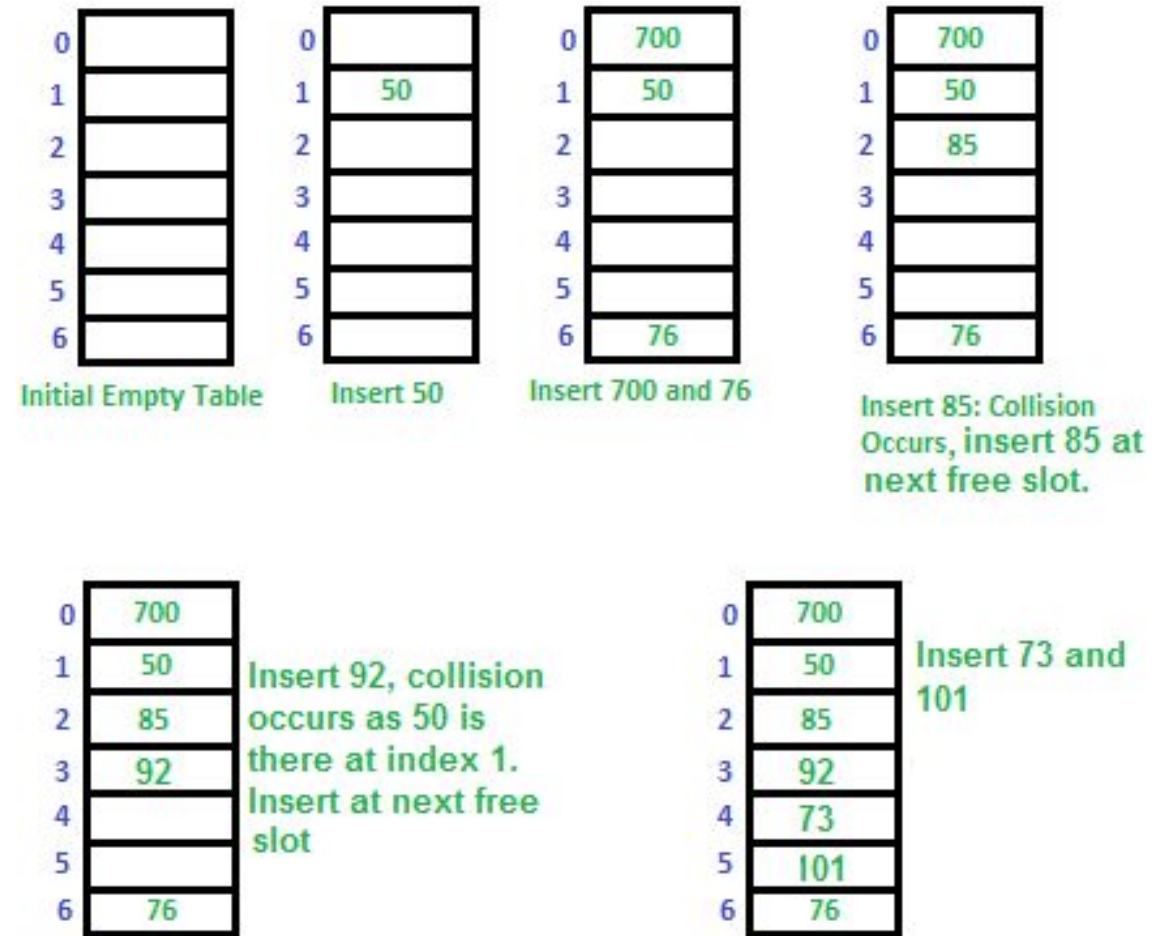
- Quadratic Probing
- Double Hashing



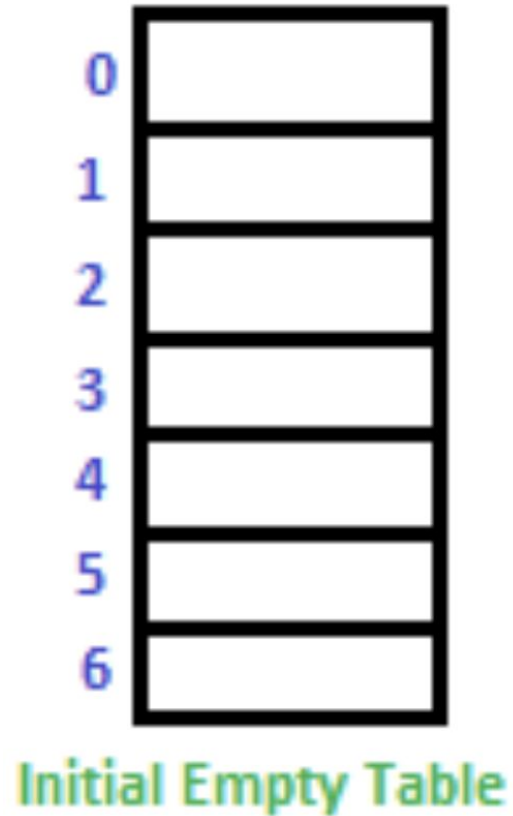
Fixing Collisions: Open Addressing

Problems:

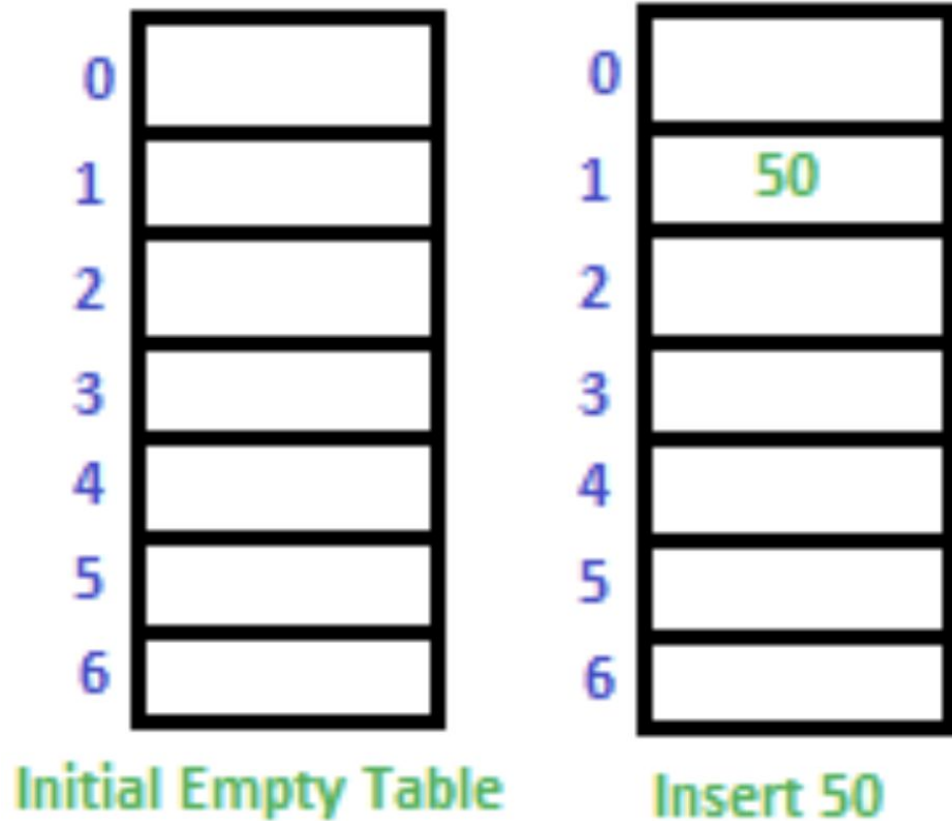
- Clustering
- Slower as table fills



Fixing Collisions: Chaining



Fixing Collisions: Chaining



Fixing Collisions: Chaining

0	
1	
2	
3	
4	
5	
6	

Initial Empty Table

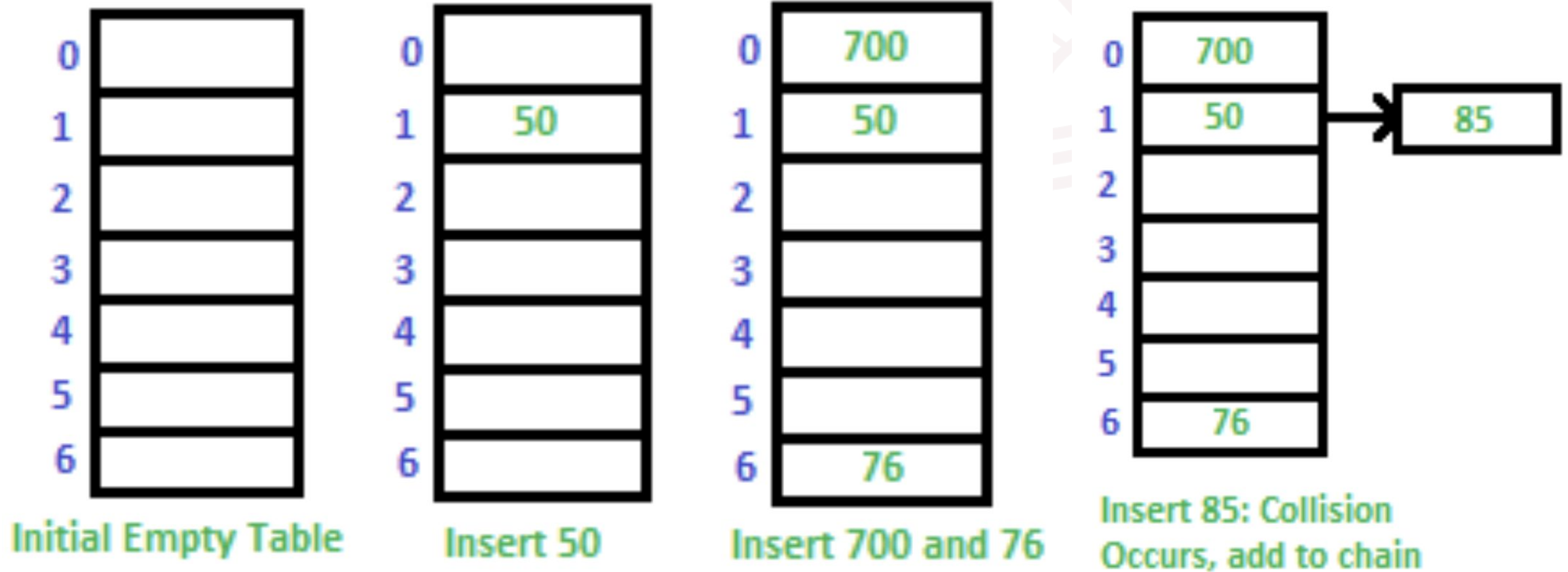
0	
1	50
2	
3	
4	
5	
6	

Insert 50

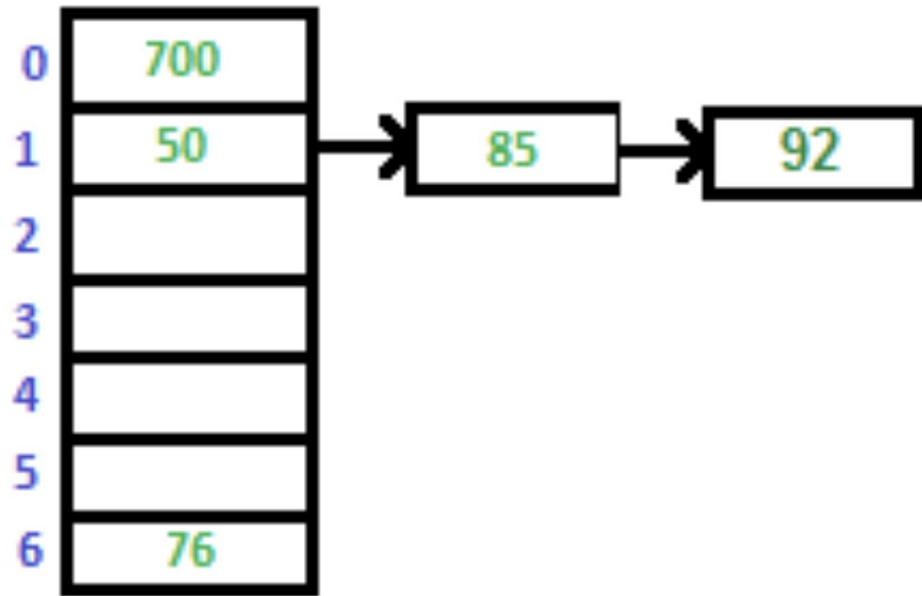
0	700
1	50
2	
3	
4	
5	
6	76

Insert 700 and 76

Fixing Collisions: Chaining

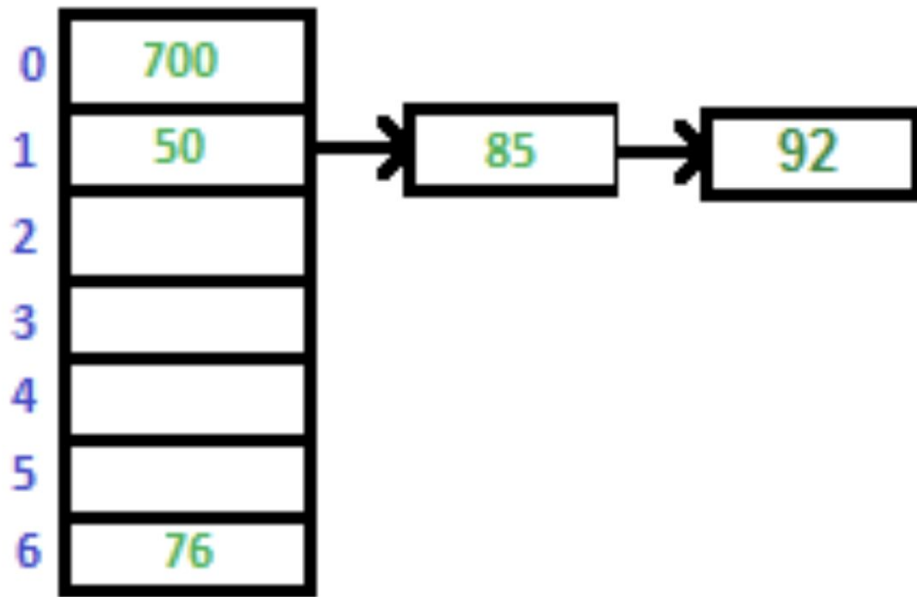


Fixing Collisions: Chaining

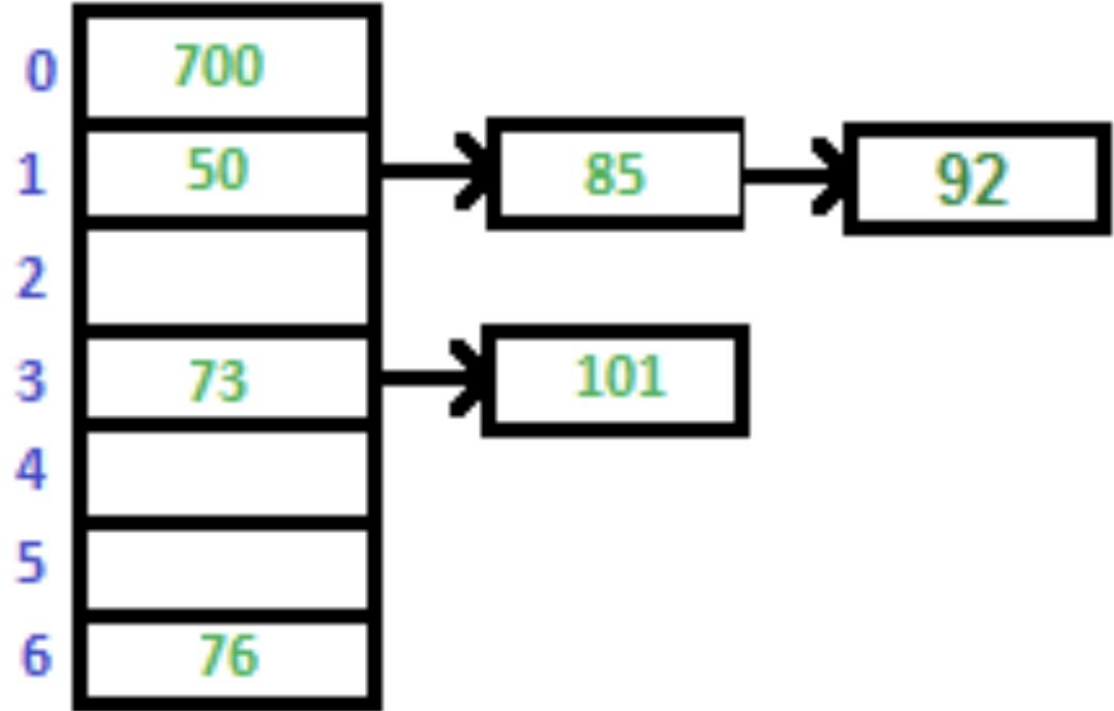


Insert 92 Collision
Occurs, add to chain

Fixing Collisions: Chaining



Insert 92 Collision
Occurs, add to chain

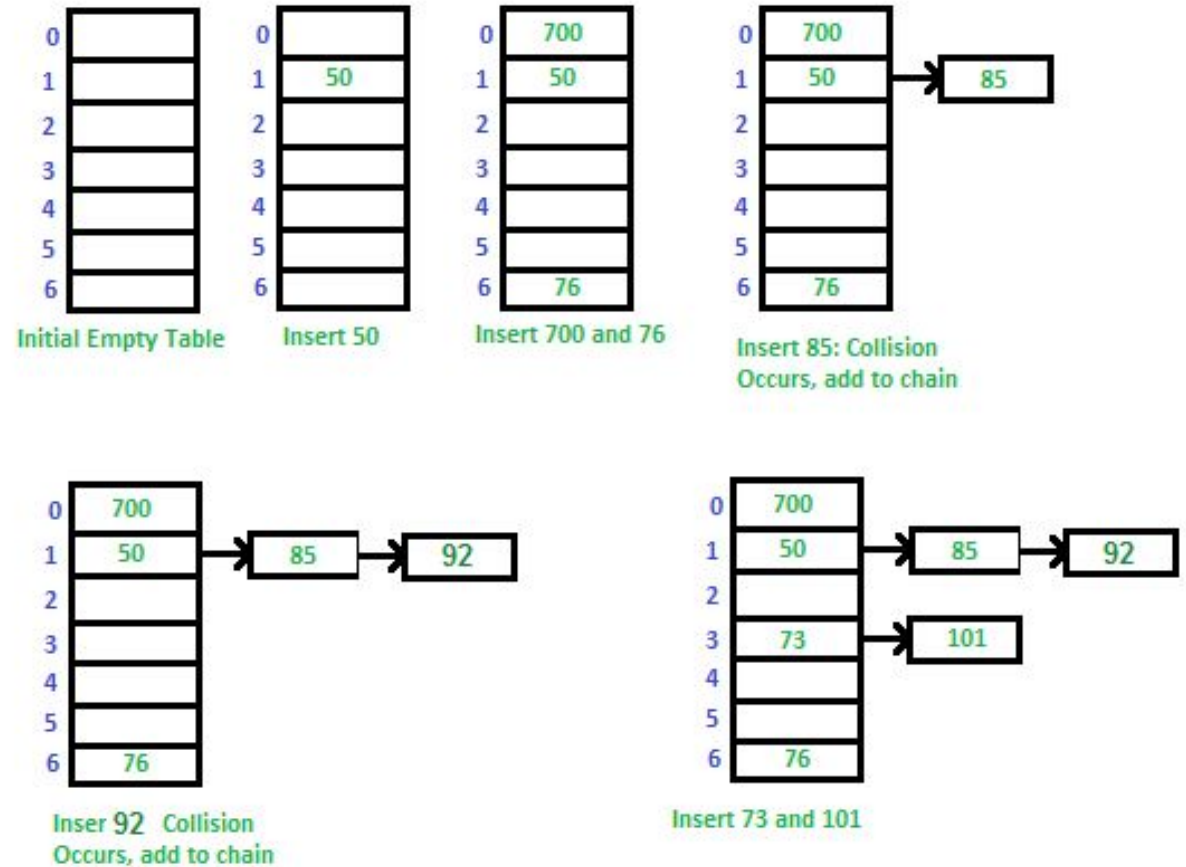


Insert 73 and 101

Fixing Collisions: Chaining

- Each index $\rightarrow$ list (bucket)
- Multiple items per slot

tl;dr: instead of avoiding collisions, we store them

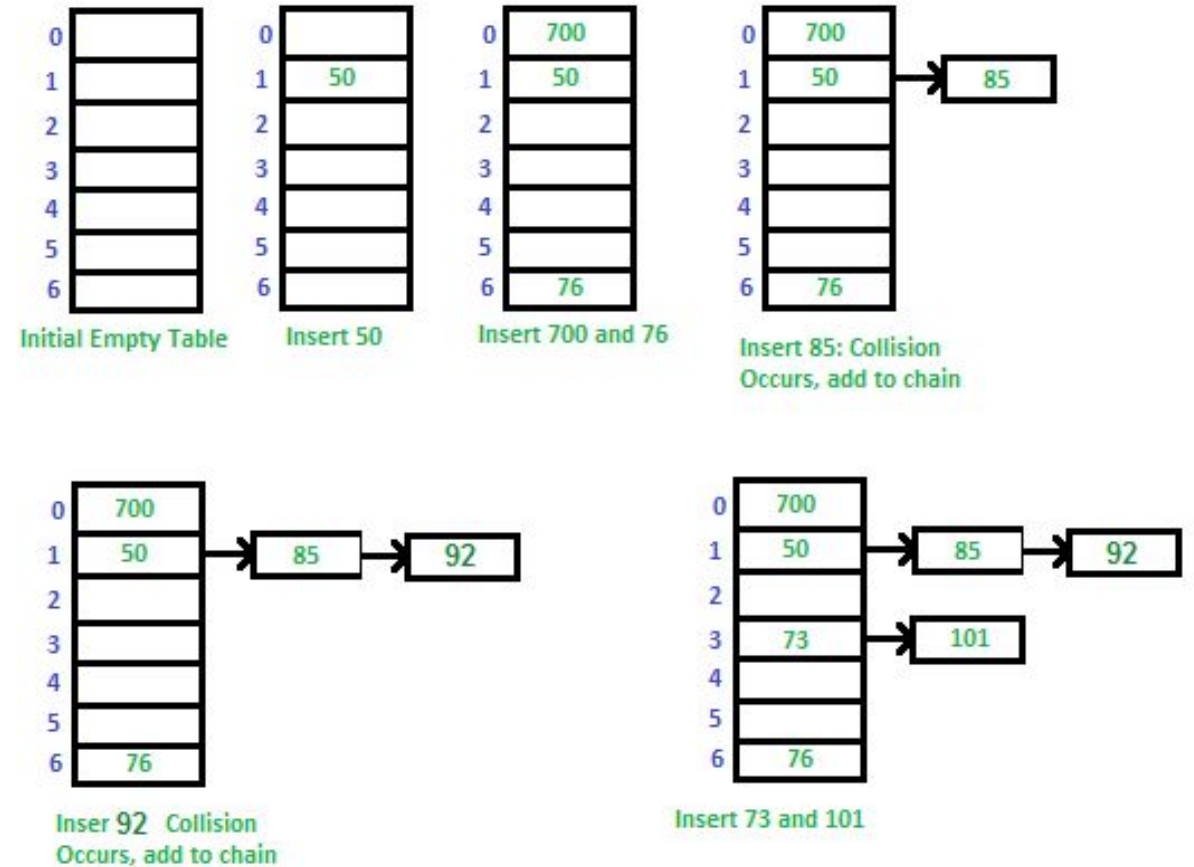


Fixing Collisions: Chaining

Chaining Tradeoffs:

- Average: $O(1)$
- Worst: $O(n)$

Worst case = everything in one bucket → back to linked list



When to Use Hash Tables

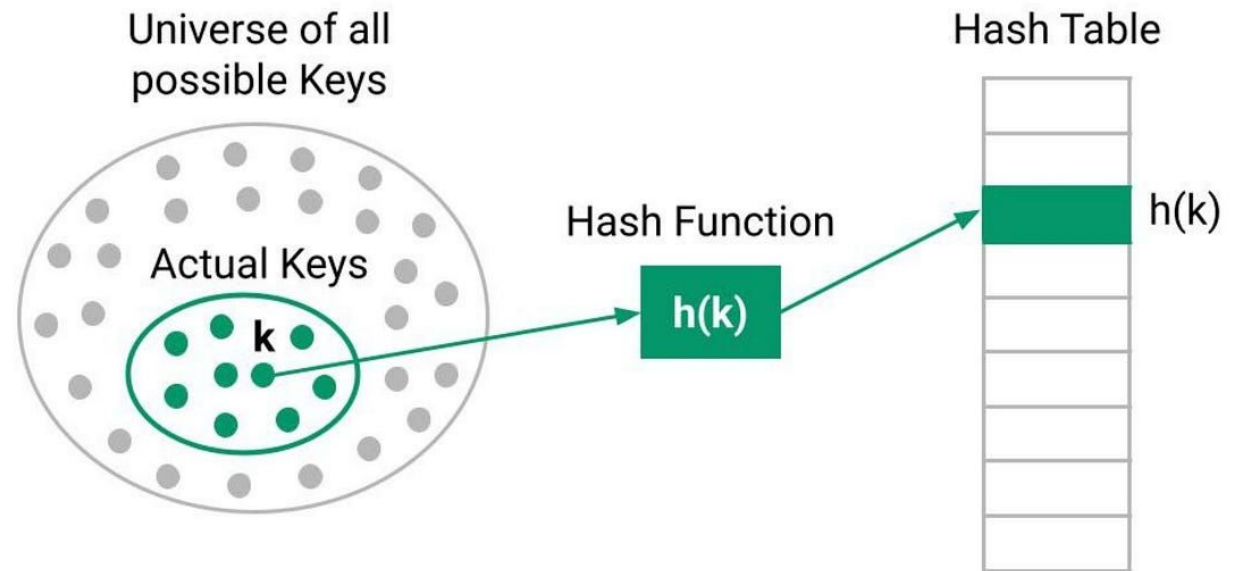
Use when:

- Fast lookup matters
- Order doesn't matter

Examples:

- Dictionaries
- Caches
- Indexing

Why do we use **Hash Tables**?



Ordered Versus Unordered Map

We're trading ordering for speed

Structure	Runtime
map (BST)	$O(\log n)$
unordered_map (hash)	$O(1)$ avg

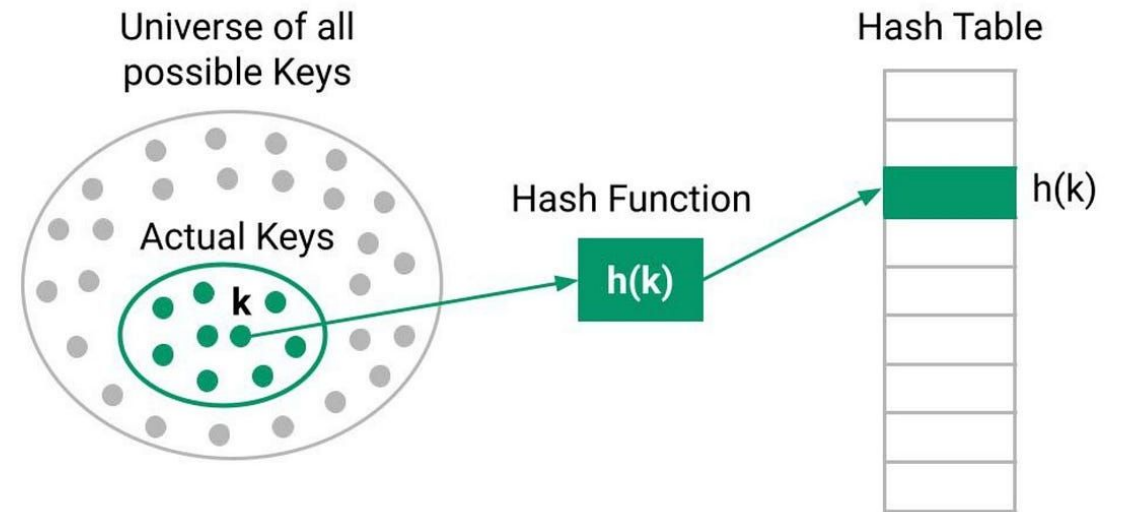
Hash Table Trainer

Now you're going to be the hash table

[Activity link](https://shorturl.at/6UVBg)

(<https://shorturl.at/6UVBg>)

Why do we use **Hash Tables**?



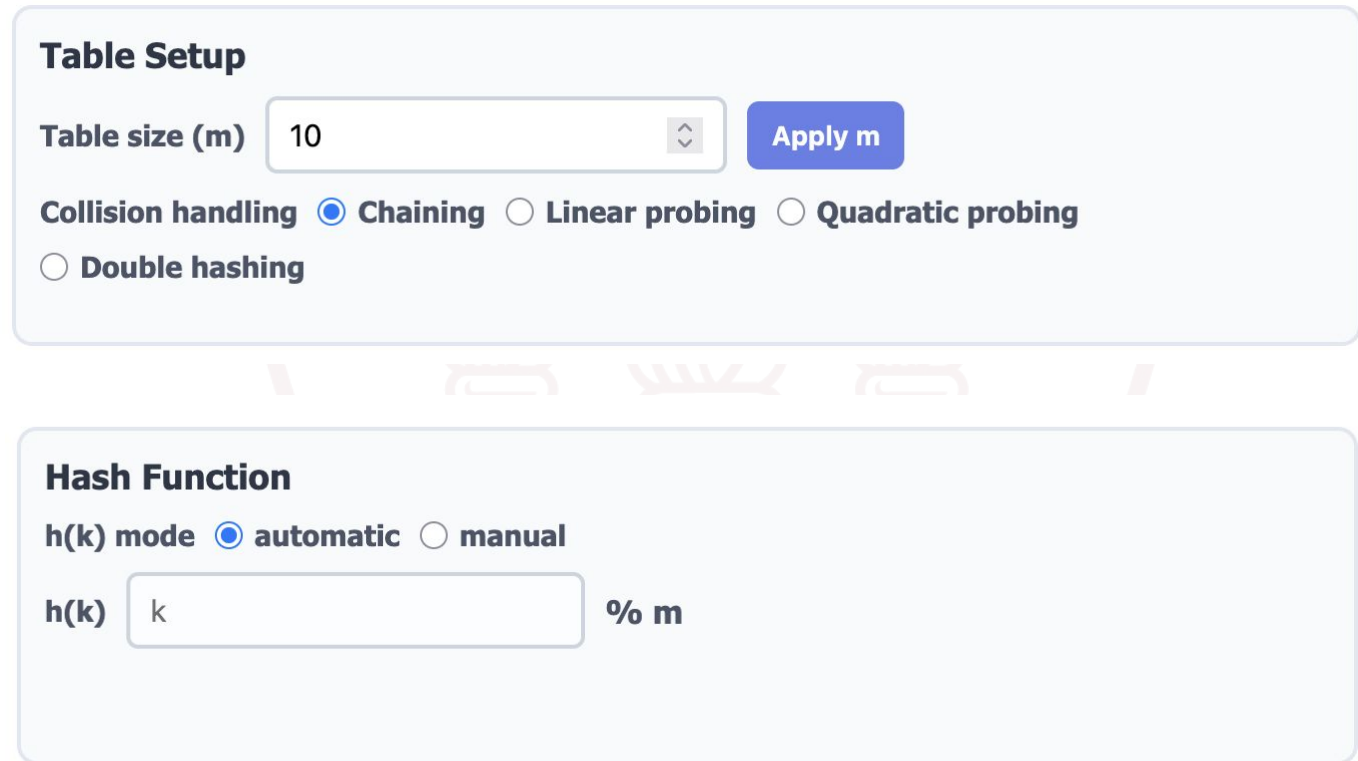
Hash Table Trainer

Task:

- Insert: 15, 26, 37, 48
- Use table size = 10
- Hash: $\text{key} \% 10$

Where does each go?

Any collisions?



The interface is divided into two main sections: 'Table Setup' and 'Hash Function'. The 'Table Setup' section includes a 'Table size (m)' dropdown set to 10, an 'Apply m' button, and 'Collision handling' options with radio buttons for 'Chaining' (selected), 'Linear probing', 'Quadratic probing', and 'Double hashing'. The 'Hash Function' section includes an 'h(k) mode' with radio buttons for 'automatic' (selected) and 'manual', and an input field for 'h(k)' containing 'k' followed by a '% m' operator.

Table Setup

Table size (m)

Collision handling ☒ Chaining ☐ Linear probing ☐ Quadratic probing
☐ Double hashing

Hash Function

h(k) mode ☒ automatic ☐ manual

h(k) % m

Hash Table Trainer – Linear Probing

Task:

- Insert: 15, 25, 35
- Use table size = 10
- Hash: $\text{key} \% 10$

What happens after collisions?

Where do they end up?

Table Setup

Table size (m) 10

Apply m

Collision handling ☐ Chaining ☒ Linear probing ☐ Quadratic probing

☐ Double hashing

Hash Function

$h(k)$ mode ☒ automatic ☐ manual

$h(k)$ k % m

The Lab

You will be implementing
an unordered set with
`string` keys using linear
probing

